



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: HERNÁNDEZ FIERROS SONIA

N° de Cuenta: 319194359

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 17 de septiembre del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1. Cohete

En este ejercicio se realizó un cohete utilizando las primitivas geométricas que se habían trabajado anteriormente y algunas nuevas. Se utilizó un cubo para formar el cuerpo, una pirámide cuadrangular para la punta, una esfera para la parte superior, un cilindro para la ventana, pirámides triangulares para las aletas y un cono para el motor.

Lo que más se trabajó fue la posición, escala y rotación de cada figura para que pudiera formar parte del cohete. Para facilitar esto se utilizó una matriz base, de manera que las diferentes piezas pudieran conservar la misma posición y rotación general.

También se utilizaron las teclas E, R y T para rotar el cohete sobre los ejes X, Y y Z.

Matriz base y transformaciones utilizadas para colocar el cuerpo y la punta del cohete

```
420 // EJERCICIO 1: COHETE
421
422 //Matriz base del cohete
423 //Aquí va la traslación al frente de la cámara y la rotación de las teclas E, R, T, para que todas las piezas del cohete se
424 glm::mat4 base = glm::mat4(1.0);
425 base = glm::translate(base, glm::vec3(0.0f, -0.2f, -6.0f));
426 base = glm::rotate(base, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
427 base = glm::rotate(base, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
428 base = glm::rotate(base, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
429
```

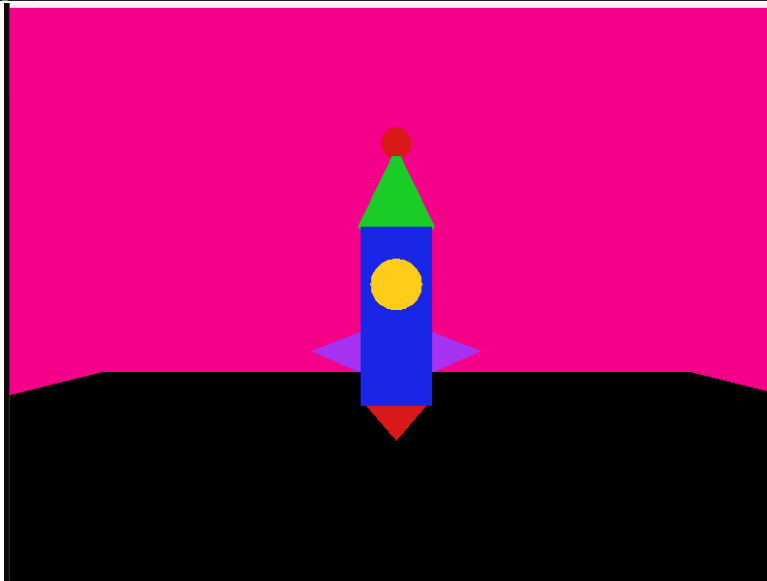
Transformaciones utilizadas para colocar las diferentes piezas que completan el cohete:

```
430 //CUERPO: cubo estirado en el eje Y
431 model = base;
432 model = glm::scale(model, glm::vec3(anchoCuerpo, altoCuerpo, anchoCuerpo));
433 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
434 color = azul;
435 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
436 meshList[0] -> RenderMesh();
437
438 //PUNTA: pirámide cuadrangular
439 //Su base local está en y=-0.5, así que el centro sube media altura para que la base caiga justo en la tapa del cuerpo (y=1.0)
440 model = base;
441 model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
442 model = glm::scale(model, glm::vec3(0.85f, 1.0f, 0.85f));
443 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
444 color = verde;
445 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
446 meshList[4] -> RenderMesh();
```

```

447 //ESFERA de hasta arriba
448 //La punta de la pirámide queda en y=2.0, la esfera se encima un poco
449 model = base;
450 model = glm::translate(model, glm::vec3(0.0f, 2.05f, 0.0f));
451 model = glm::scale(model, glm::vec3(0.18f, 0.18f, 0.18f));
452 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
453 color = rojo;
454 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
455 sp.render();
456
457 //VENTANA: cilindro acostado sobre la cara del frente
458 //El cilindro nace parado en Y, se gira 90 grados en X para que su eje apunte en Z
459 model = base;
460 model = glm::translate(model, glm::vec3(0.0f, 0.35f, 0.50f));
461 model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
462 model = glm::scale(model, glm::vec3(0.28f, 0.14f, 0.28f));
463 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
464 color = amarillo;
465 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
466 meshList[2]->RenderMeshGeometry();
467
468 //ALETA DERECHA: pirámide triangular
469 //Gira -90 grados en Z para que la punta apunte hacia +X
470 model = base;
471 model = glm::translate(model, glm::vec3(0.55f, -0.45f, 0.0f));
472 model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
473 model = glm::scale(model, glm::vec3(0.8f, 1.0f, 0.8f));
474 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
475 color = morado;
476 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
477 meshList[1]->RenderMesh();
478
479 //ALETA IZQUIERDA: misma pirámide, girada al otro lado
480 model = base;
481 model = glm::translate(model, glm::vec3(-0.55f, -0.45f, 0.0f));
482 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
483 model = glm::scale(model, glm::vec3(0.8f, 1.0f, 0.8f));
484 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
485 color = morado;
486 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
487 meshList[1]->RenderMesh();
488
489 //MOTOR: cono de abajo
490 //El cono nace con la punta hacia arriba, se gira 180 grados en X para voltearlo
491 model = base;
492 model = glm::translate(model, glm::vec3(0.0f, -1.25f, 0.0f));
493 model = glm::rotate(model, 180 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
494 model = glm::scale(model, glm::vec3(0.42f, 0.5f, 0.43f));
495 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
496 color = rojo;
497 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
498 meshList[3]->RenderMeshGeometry();
499

```



Ejercicio 2. Ocho pirámides

En el segundo ejercicio se formó una figura utilizando ocho pirámides cuadrangulares. Para colocarlas se utilizó una matriz base y a partir de ella se fueron haciendo diferentes traslaciones, rotaciones y escalas.

Uno de los problemas de este ejercicio fue acomodar correctamente las ocho pirámides para que quedaran juntas y con la orientación solicitada. Fue necesario cambiar varias veces los valores de las posiciones y rotaciones hasta conseguir una distribución adecuada.

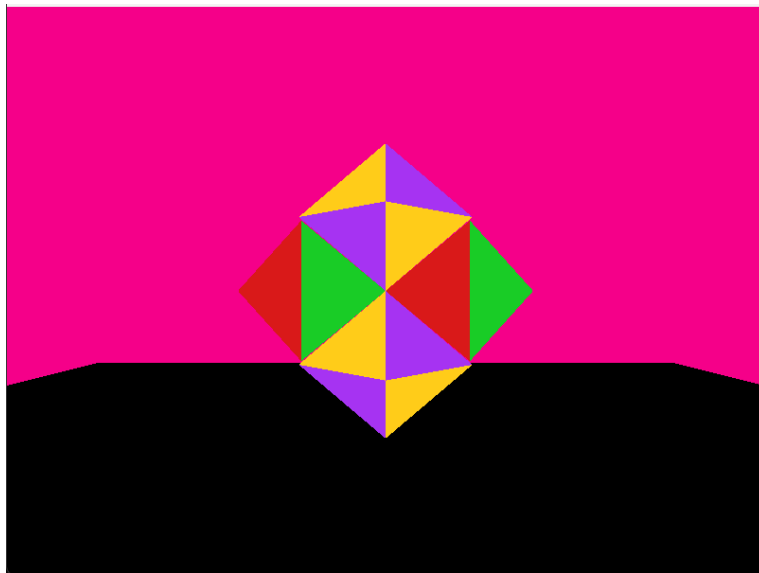
También fue necesario hacer una modificación para poder mostrar diferentes colores en las caras de las pirámides. Para esto se separaron las caras de la pirámide en diferentes mesh, ya que de esta manera se podía cambiar el color antes de dibujar cada cara.

```
140 //La misma pirámide cuadrangular pero partida en sus 5 caras.
141 //Se separa porque el shader manda un solo color por cada RenderMesh(),
142 //así cada cara se dibuja aparte y puede llevar su propio color.
143 void CrearPiramideCuadrangularPorCaras()
144 {
145     unsigned int tri_idx[] = { 0, 1, 2 };
146
147     // 1. CARA FRONTAL (Verde)
148     GLfloat v_front[] = { 0.5f,-0.5f,0.5f, -0.5f,-0.5f,0.5f, 0.0f,0.5f,0.0f };
149     Mesh* mFront = new Mesh(); mFront->CreateMesh(v_front, tri_idx, 9, 3);
150     meshList.push_back(mFront);
151
152     // 2. CARA IZQUIERDA (Morada)
153     GLfloat v_left[] = { -0.5f,-0.5f,0.5f, -0.5f,-0.5f,-0.5f, 0.0f,0.5f,0.0f };
154     Mesh* mLeft = new Mesh(); mLeft->CreateMesh(v_left, tri_idx, 9, 3);
155     meshList.push_back(mLeft);
156
157     // 3. CARA TRASERA (Amarilla)
158     GLfloat v_back[] = { -0.5f,-0.5f,-0.5f, 0.5f,-0.5f,-0.5f, 0.0f,0.5f,0.0f };
159     Mesh* mBack = new Mesh(); mBack->CreateMesh(v_back, tri_idx, 9, 3);
160     meshList.push_back(mBack);
161
162     // 4. CARA DERECHA (Roja)
163     GLfloat v_right[] = { 0.5f,-0.5f,-0.5f, 0.5f,-0.5f,0.5f, 0.0f,0.5f,0.0f };
164     Mesh* mRight = new Mesh(); mRight->CreateMesh(v_right, tri_idx, 9, 3);
165     meshList.push_back(mRight);
166
167     // 5. BASE CUADRADA (Azul)
168     unsigned int quad_idx[] = { 0, 1, 2, 0, 2, 3 };
169     GLfloat v_base[] = { 0.5f,-0.5f,0.5f, 0.5f,-0.5f,-0.5f, -0.5f,-0.5f,-0.5f, -0.5f,-0.5f,0.5f };
170     Mesh* mBase = new Mesh(); mBase->CreateMesh(v_base, quad_idx, 12, 6);
171     meshList.push_back(mBase);
172 }
```

```

517 //Pirámide 1 (centro izquierda, muestra la cara verde)
518 model = glm::translate(base, glm::vec3(-0.3f, 0.0f, 0.0f));
519 model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
520 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
521 DibujarPiramideMulticolor(model, uniformModel, uniformColor);
522
523 //Pirámide 2 (centro derecha, rotada para mostrar la cara roja)
524 model = glm::translate(base, glm::vec3(0.3f, 0.0f, 0.0f));
525 model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
526 model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
527 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
528 DibujarPiramideMulticolor(model, uniformModel, uniformColor);

```



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El principal problema que tuve fue acomodar correctamente las figuras geométricas. Al principio entendía de manera general cómo funcionaban las traslaciones, rotaciones y escalas, pero al utilizar varias figuras al mismo tiempo era más complicado calcular dónde debía quedar cada una.

En el caso del cohete, fue necesario modificar varias veces las posiciones de las piezas para que quedaran juntas y no estuvieran separadas o dentro de otras figuras. También tuve que tener en cuenta que algunas figuras tenían una posición inicial diferente, por lo que no siempre bastaba con aplicar la misma transformación.

Otro problema fue utilizar las nuevas primitivas, principalmente la esfera, el cilindro y el cono, ya que anteriormente se habían utilizado principalmente cubos y pirámides. Fue necesario entender primero cómo estaba orientada cada figura para poder colocarla correctamente. Por ejemplo, el cilindro tuvo que rotarse para utilizarlo como la ventana del cohete y el cono tuvo que rotarse para utilizarlo como motor.

En el ejercicio de las ocho pirámides, el problema principal fue encontrar las posiciones y rotaciones adecuadas para que las ocho piezas quedaran acomodadas. Además, cada cara debía tener un color diferente, por lo que no era suficiente con dibujar una pirámide completa con un solo color. Para resolverlo se separaron las cinco caras de la pirámide y se dibujó cada una por separado cambiando el color.

Los problemas se fueron resolviendo modificando poco a poco los valores de translate, rotate y scale, y observando el resultado directamente en la ventana del programa.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

La parte más sencilla fue utilizar las figuras que ya se habían trabajado anteriormente, como el cubo y las pirámides. La dificultad aumentó al utilizar figuras nuevas como el cilindro, el cono y la esfera, ya que primero fue necesario entender su orientación y después aplicar las transformaciones necesarias.

El ejercicio de las ocho pirámides también tuvo una mayor dificultad porque se tenían que acomodar varias figuras al mismo tiempo y además controlar el color de cada una de sus caras.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Hubiera sido útil dedicar un poco más de tiempo a las nuevas primitivas, como el cilindro, el cono y la esfera, en clase me costó entender cómo es que se crean.

c. Conclusión.

La práctica me ayudó a entender mejor cómo se pueden utilizar figuras geométricas sencillas para construir objetos más completos. También pude practicar las transformaciones de traslación, rotación y escala, y observar cómo un pequeño cambio en sus valores puede modificar bastante la posición final de un objeto.

La parte que más trabajo me dio fue acomodar correctamente las diferentes figuras, pero al ir modificando las transformaciones y observando los resultados pude llegar a la composición solicitada. También entendí mejor que para conseguir diferentes colores en las caras de una figura puede ser necesario separar sus caras y dibujarlas individualmente.

1. Bibliografía en formato APA

Khronos Group. (s. f.). *OpenGL Wiki*. <https://wikis.khronos.org/opengl/>
G-Truc Creation. (s. f.). *OpenGL Mathematics (GLM)*. <https://github.com/g-truc/glm>